

# Q-Care Detect / LAKSHYA — Project explanation

SIH 2026 problem SIH26139 (Egreen Quanta). Written 2026-09-09. Research and education only.

Research risk classification - not for clinical use.

---

## 1. What this project is

This repo is a **hybrid quantum–classical machine learning research platform** for public biomedical **benchmark tables**. The assignment theme is early disease detection; the software does **research risk classification on one table at a time**. It is not a medical device, not a diagnosis tool, and not a symptom checker. The public UI is branded **LAKSHYA** (stitchfrontend/).

Typical flow: load a public table → classical cleaning, stratified 80/20 split (seed 42), scaling and feature selection → train classical baselines and small simulator QML models on the same reduced columns → compare F1, AUC, runtime, and paired statistics. Each model only sees that table's columns and one label.

## 2. Are we using QML, classical ML, or hybrid ML?

We use **hybrid ML**: most of the pipeline is classical; a **small quantum (or quantum-inspired) block** runs on a **classical simulator** (Qiskit Aer statevector). We are **not** running a hospital quantum computer, and we are **not** training a quantum model on 30 raw Wisconsin columns or on images.

### *Why hybrid (not “pure QML”)*

Near-term circuits have few qubits and shallow depth. Encoding dozens of raw labs into a circuit is a poor fit. So classical code reduces the table to about **4–8 selected features** (one qubit per feature in v1). That is the hybrid split: computers do load, clip, scale, SelectKBest, and evaluation; the quantum library builds a small circuit or kernel on those k numbers.

### *Why not claim “QML is more accurate”*

On Wisconsin on this machine, **classical logistic regression beat VQC/QSVC on F1**. QSVC was statistically **indistinguishable** from an RBF SVM on the same k columns (McNemar  $p = 1.000$ ). VQC/QNN were worse. That is an acceptable research outcome. Pitch **architecture and compute cost**, not a quantum accuracy win.

## 3. How the hybrid models work

Shared front end for QML v1: train-only feature selection (typical kept Wisconsin features include mean perimeter, mean concave points, worst radius, worst perimeter, worst area, worst concave points). Values are scaled. Then one of:

### *Quantum kernel SVM (QSVC)*

A **ZZFeatureMap** encodes the k numbers as rotation angles and adds ZZ entangling gates (reps = 2 by default). **FidelityStatevectorKernel** computes the overlap of those quantum states (exact simulator math, not noisy shots — shot-based kernels hung here). A classical SVM then uses that kernel matrix like any other kernel SVM. The honest control is an **RBF SVM on the same k columns**: if QSVC  $\approx$  RBF, the quantum kernel is not buying extra accuracy on this table.

### *Variational quantum classifier (VQC)*

Same feature map, plus a trainable **RealAmplitudes** ansatz. A classical optimizer (**COBYLA**, maxiter 60 by default) updates circuit parameters to separate the two classes. This is hybrid because the cost function is

evaluated from the circuit, but the optimizer and data pipeline are classical. VQC is slow on the simulator compared with logistic regression.

### ***Optional QNN***

**EstimatorQNN + NeuralNetworkClassifier** (fit on  $\pm 1$  labels). Ranking uses raw expectation values, not calibrated probabilities. Optional via `python -m src.train_qml --with-qnn`.

### ***What stays classical***

- Logistic regression and random forest on full or reduced tables.
- Large tables (cardio, BRFSS, etc.) stay classical; QSVC refuses tables over 5000 rows.
- Optional laptop CNNs (WBC / MSK images) are PyTorch, not QML.
- Decision support = probability band + threshold on **one** label only.

## **4. Technology stack**

### ***Data and classical ML***

- Python 3, pandas, NumPy, scikit-learn, joblib, matplotlib (Agg backend).
- Public tables (sklearn Wisconsin, UCI/Kaggle-style sets). Duplicate rows dropped by default.

### ***Hybrid QML (simulator)***

- Qiskit, Qiskit Aer, Qiskit Machine Learning, Qiskit Algorithms.
- No real QPU in this delivery; Aer statevector is allowed.

### ***Optional deep learning***

PyTorch / torchvision for small image experiments when CUDA is available. Not fused with tabular QML into one diagnosis.

### ***Apps and API***

- **LAKSHYA**: Vite + React + TypeScript + Tailwind in `stitchfrontend/`; FastAPI serves the production build.
- **API**: FastAPI + Uvicorn — catalog, research-score, paste-a-record, interview, locked PDF.
- **Streamlit**: research dashboard (`app/streamlit_app.py`).
- **PDF**: reportlab + pypdf (generated open-password; name/DOB are labels only).

### ***Optional cloud pieces***

- NVIDIA NIM (OpenAI-compatible) to extract `field: number` pairs into one selected table — not a disease picker.
- Supabase can store a fingerprint of PDF unlock keys; the PDF still works without it.

## **5. What we refuse (product boundaries)**

- No fused “what disease do I have?” score across tables.
- No general symptom checker or invented ICD lists.

- No ranking of named doctors; specialty **domain hints** only.
- No concatenating duplicate Wisconsin copies to fake more patients.

## 6. How to run (short)

Activate .venv. Train: `python -m src.train_classical, python -m src.train_phase2, python -m src.train_qml --with-qnn`. UI: `cd stitchfrontend; npm install; npm run build` then `uvicorn src.api_server:app --reload --port 8000`. Tests: `pytest`.

---

Regenerate this file: `python -m src.generate_explanation_pdf`